



OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 9

Team No.: 3

Project Title: Flight Simulator and Monitoring System

Team Members

Roll Number	Student Name	Signature
2520030359	Mohammad Faizaan	
2520030267	Ishaan Kulkarni	

1. Abstract

Flight Simulator and Monitoring System is a Linux-based Operating Systems project that demonstrates how multiple processes can communicate, coordinate, and respond to changing conditions in a simulated flight environment. A flight simulation requires continuous generation and monitoring of parameters such as altitude, speed, fuel level, engine status, and flight condition. Managing these activities concurrently requires suitable process communication, synchronization, monitoring, and control mechanisms.

The project focuses on key Operating Systems and system programming concepts such as process creation, process identification, inter-process communication (IPC), signal generation and handling, process synchronization, concurrency, file I/O, and process control. Separate components can be used for flight simulation, monitoring, control, and logging, with Linux mechanisms such as `fork()`, `exec()`, `pipe()`, FIFO-based communication, `signal()/sigaction()`, `kill()`, `wait()`, POSIX threads, mutexes, and file-I/O system calls supporting the system.

The proposed solution consists of a Linux-based C/C++ application in which independent processes or threads perform simulation, monitoring, control, and logging tasks. The expected outcome is a functional demonstration of how an operating system coordinates concurrent activities in a real-world-inspired application. The project also provides practical understanding of process management, IPC, synchronization, concurrency, and system-level programming through a flight simulation and monitoring scenario.

2. Problem Statement

A flight simulation and monitoring application has to manage several activities at the same time. The simulator must continuously update flight parameters, the monitoring component must observe those values, the control component must respond to important events, and the system must preserve operational information for later analysis. Without proper process communication and synchronization, data may become inconsistent, events may not be handled correctly, and concurrent operations may interfere with one another.

This project addresses the problem of efficiently coordinating simulation, monitoring, control, and logging activities using Linux Operating Systems mechanisms. Separate processes or threads perform different functions, while IPC mechanisms provide controlled data exchange. Signals can be used for asynchronous notifications and important events, while synchronization mechanisms help coordinate shared operations and safe termination.

The project is relevant to Operating Systems and Systems Programming because it demonstrates practical concepts including process creation, process control, IPC, signals, synchronization, file handling, and concurrent execution.

3. Objectives

1. To design a Linux-based flight simulation application using Operating Systems and systems-programming concepts.
2. To create and manage simulator, monitoring, control, and logging processes or threads.
3. To implement IPC mechanisms for transferring flight telemetry and control information.
4. To use signals or asynchronous notifications for warnings, emergency events, pause/resume, and safe termination.
5. To demonstrate synchronization and concurrent execution while preventing inconsistent access to shared resources.
6. To store and retrieve flight records using Linux file-I/O mechanisms.
7. To provide a practical demonstration of how an Operating System coordinates concurrent real-world-inspired tasks.

4. Proposed Methodology

1. Process Creation: The program starts with a main process that creates or launches the simulator, monitoring, control, and logging components.
2. Process Identification: Each process obtains its PID and PPID so that components can be identified and managed.
3. Flight Simulation: The simulator continuously generates or updates altitude, airspeed, fuel level, engine status, and other flight parameters.
4. Process Communication: Telemetry and control information are exchanged through suitable IPC mechanisms such as anonymous pipes, named pipes (FIFOs), or shared memory.
5. Monitoring: The monitoring component receives flight data, displays the current state, checks defined thresholds, and records important events.
6. Event Handling: Signals are used for asynchronous notification and control actions such as emergency alerts, pause/resume requests, or termination.
7. Synchronization: Mutexes, condition variables, semaphores, or wait-based process synchronization are used where required to coordinate concurrent activities.
8. Logging: Flight telemetry and important events are written to log files using Linux file-I/O system calls.
9. Safe Termination: The parent or control component monitors the system and ensures that processes, files, and communication resources are closed safely.

4. Proposed Methodology (Execution Flow)

Execution Flow: Initialize System → Create Processes → Generate Flight Telemetry → Transfer Data through IPC → Monitor Flight State → Detect Events → Send Control/Alert Signals → Synchronize Operations → Log Flight Data → Terminate Processes Safely.

The design follows the course handout's systems-programming areas: process creation and control, IPC through pipes/FIFOs/signals, Linux file I/O, and POSIX concurrency primitives.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Process Creation – fork(), exec()	Creates and launches the simulator, monitoring, control, or logging processes.
Process Identification – getpid(), getppid()	Identifies processes so the system can display and manage the simulation components.
IPC – pipe(), FIFO	Transfers telemetry and control information between processes.
Signal Handling – signal(), sigaction(), kill()	Provides asynchronous notification and control for warnings, emergency events, pause/resume, and termination.
Synchronization – wait(), mutexes, semaphores	Coordinates process/thread completion and protects shared resources during concurrent operations.
File I/O – open(), read(), write(), close()	Stores flight telemetry, event messages, and monitoring records in log files.
POSIX Threads – pthreads	Supports concurrent monitoring, display, logging, or event-handling tasks where appropriate.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030359	Mohammad Faizaan	Flight simulation module, process creation/control, generation of flight parameters, and simulator-side integration.
2520030267	Ishaan Kulkarni	Monitoring and logging module, IPC data handling, event monitoring, flight-status display, and testing/integration support.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Provides the operating environment for development, compilation, execution, and monitoring.
C / C++	Used to implement processes, IPC, signal handling, synchronization, file operations, and simulation logic.
GCC / G++ Compiler	Compiles the C/C++ source code into executable programs.
POSIX Libraries	Provides APIs for processes, signals, threads, synchronization, and system-level operations.
Linux Terminal / Shell	Used to compile, execute, test, observe process behavior, and inspect logs.
GDB / Valgrind / strace	Supports debugging, memory-error analysis, and system-call tracing during development and testing.

8. Expected Outcome

1. The system will successfully initialize and run the flight simulation and monitoring components.
2. The simulator will generate changing flight parameters and make them available to the monitoring component.
3. IPC mechanisms will allow telemetry and control information to move between relevant processes.
4. The monitoring component will display flight status and identify defined warning or abnormal conditions.
5. Signals will demonstrate asynchronous event notification and control actions such as pause/resume or emergency handling.
6. Concurrent activities will be coordinated using suitable synchronization mechanisms.
7. Flight telemetry and significant events will be stored in log files using Linux file-I/O operations.
8. The system will terminate processes and close resources safely.
9. Overall, the project will provide practical understanding of process management, IPC, signals, file handling, and concurrency in a Linux environment.

9. GitHub Repository

GitHub Repository Link:	
Repository Created:	Yes No

Faculty Name and Signature

Faculty Name:	
Signature:	
Remarks:	

Faculty Use

Project Approved:	Yes No
Date:	_____
Faculty Signature:	

Project Title: Flight Simulator and Monitoring System

Section: 9 Team: 3